

Formalização de diferentes noções de ordenação

Carolina Campos - 221030830

Gustavo Rinaldi - 222008664

12 de julho de 2026

1 Introdução

Equivalência entre diferentes noções de ordenação. O objetivo deste arquivo é formalizar e demonstrar a equivalência entre quatro definições distintas de listas ordenadas.

2 Desenvolvimento

2.1 Definições de ordenação e propriedades básicas

2.1.1 Ordenação 1 (ord1)

A primeira definição de ordenação, chamada **ord1**, é uma definição indutiva contendo 3 regras de formação:

$$\frac{}{ord1\ nil} (ord1_nil) \quad \frac{}{ord1\ (x :: nil)} (ord1_one) \quad \frac{x \leq y \quad ord1(y :: l)}{ord1\ (x :: y :: l)} (ord1_all)$$

```
Inductive ord1 : list nat → Prop :=  
| ord1_nil: ord1 nil  
| ord1_one: ∀ x, ord1 (x::nil)  
| ord1_all: ∀ l x y, x ≤ y → ord1 (y::l) → ord1 (x::y::l).
```

2.1.2 Ordenação 2 (ord2)

A segunda definição de ordenação, chamada **ord2**, é uma definição indutiva contendo 2 regras de formação:

$$\frac{}{ord2\ nil} (ord2_nil) \quad \frac{x \leq^* l \quad ord2\ l}{ord2\ (x :: l)} (ord2_all)$$

onde $x \leq^* l$ significa que x é menor ou igual que todo elemento da lista l . Formalmente, este predicado é definido a seguir:

Definition $le_all\ x\ l := \forall y, In\ y\ l \rightarrow x \leq y$.

```
Inductive ord2 : list nat → Prop :=  
| ord2_nil: ord2 nil  
| ord2_all: ∀ l x, x ≤* l → ord2 l → ord2 (x::l).
```

2.1.3 Ordenação 3 (ord3)

A terceira definição de ordenação, chamada **ord3**, diz que uma lista está ordenada se cada elemento é menor ou igual ao elemento seguinte:

Definition *ord3* (*l* : *list nat*) : **Prop** :=

$\forall i, \text{length } l > 1 \rightarrow (S\ i) < \text{length } l \rightarrow \text{nth } i\ l\ 0 \leq \text{nth } (S\ i)\ l\ 0.$

Lemma *ord3_nil*: *ord3 nil*.

Lemma *ord3_one*: $\forall x, \text{ord3 } (x::\text{nil}).$

Lemma *ord3_two*: *ord3* (*1::2::nil*).

2.1.4 Ordenação 4 (ord4)

A quarta definição de ordenação, chamada **ord4**, diz que uma lista está ordenada se cada elemento é menor ou igual que qualquer elemento que esteja em posição anterior:

Definition *ord4* (*l* : *list nat*) : **Prop** :=

$\forall i\ j, i < j \rightarrow j < \text{length } l \rightarrow \text{nth } i\ l\ 0 \leq \text{nth } j\ l\ 0.$

Lemma *ord4_nil*: *ord4 nil*.

2.2 Lemas auxiliares de equivalência

2.2.1 Lemas para a prova de **ord1_equiv_ord2**

Para provarmos a equivalência entre as definições indutivas **ord1** e **ord2** (**ord1** $\leftrightarrow$ **ord2**), trataremos uma implicância de cada vez. Primeiro **ord1** $\rightarrow$ **ord2** e, em seguida, **ord2** $\rightarrow$ **ord1**.

Lema **ord1_to_ord2** prova **ord1** $\rightarrow$ **ord2**, ou seja, que toda lista **ord1** também é **ord2**. A base utilizada é baseada na divisão de **ord1** em três regras de formação, executada pela tática **induction** *H*, que divide a prova em três casos:

1. Caso **nil**: utilizada a tática **apply** **ord2_nil**, que prova que uma lista vazia pelas regras de **ord1** também é vazia pelas regras de **ord2**.

2. Caso de um elemento (*x* :: **nil**): prova feita com **ord2_all**, que exige que

- *x* seja menor ou igual a todos os elementos do resto da lista ($x \leq^* \text{nil}$). Como a lista **nil** não tem elementos, isso é uma verdade vazia. A tática **inversion** *Hy* verifica que não faz sentido ter elementos em **nil** e finaliza o problema.
- o resto da lista esteja ordenado (**ord2 nil**), o que é resolvido com **apply** **ord2_nil**.

3. Caso de dois ou mais elementos (*x* :: *y* :: *l*): Sabe-se por hipótese que $x \leq y$ e que o resto da lista (*y* :: *l*) está ordenado. Logo, provar **ord2** (*x* :: *y* :: *l*). Isso se faz ao provar que *x* é menor ou igual a todos os elementos de *y* :: *l* ($x \leq^* y :: l$) quando se aplica **ord2_all**. Isso é possível com auxílio de um elemento qualquer (*z*) dessa lista. Ele pode ser duas coisas e resolvido com **destruct** *H**z*

- Ele é o próprio *y*: Se *z* é *y*, a hipótese $x \leq y$ já resolve o problema (**subst. assumption.**).
- Ele está dentro de *l*: Aqui depende de hipótese de indução (*IHord1*), que garante que *y* :: *l* já é **ord2**. Usando **inversion** sobre essa hipótese, *y* é menor que todos os elementos de *l* ($y \leq z$). Como $x \leq y$ e $y \leq z$, a tática matemática *lia* usa transitividade para concluir que $x \leq z$.

Lemma *ord1_to_ord2* : $\forall l, \text{ord1 } l \rightarrow \text{ord2 } l.$

Lema `ord2_to_ord1` prova o contrário $\text{ord2} \rightarrow \text{ord1}$. Como a `ord2` só possui duas regras de formação, a indução gera apenas dois casos:

1. Caso `nil`: Resolvido com `apply ord1_nil`.
2. Caso de um ou mais elementos ($x :: l$): Sabe-se que $x \leq^* l$ e que, por hipótese, a lista l atende aos critérios da definição de `ord1`. O problema é resolver o fato de `ord1` ter regras diferentes para listas com 1 e listas com 2+ elementos. Por isso, a tática `destruct l as [| y l']` analisa dois subcasos da própria lista l
 - Se l for `nil`: Então a lista original é apenas x , resolvido com a regra `apply ord1_one`.
 - Se l tem elementos ($y :: l'$): Então a lista original é $x :: y :: l'$. A regra que funciona para isso é `ord1_all`, que exige que
 - $x \leq y$: Como x é menor ou igual a todos os elementos da lista $y :: l'$, basta aplicar `apply H` apontando que y é o primeiro elemento da lista (`left. reflexivity.`).
 - $y :: l'$ está ordenado em `ord1`: a hipótese de indução já assume isso (`assumption.`).

Lemma `ord2_to_ord1` : $\forall l, \text{ord2 } l \rightarrow \text{ord1 } l$.

2.2.2 Lemas para a prova de `ord1_equiv_ord3`

Para provarmos a equivalência entre as definições indutivas `ord1` e `ord3` ($\text{ord1} \leftrightarrow \text{ord3}$), trataremos uma implicância de cada vez. Primeiro $\text{ord1} \rightarrow \text{ord3}$ e, em seguida, $\text{ord3} \rightarrow \text{ord1}$.

Lema `ord1_to_ord3` é baseado em indução sobre a hipótese de ordenação H , porque `ord1` é uma definição indutiva. Por isso, `intros l H. induction H` inicia a prova trazendo a hipótese de que a lista é `ord1`. A indução cria três casos exatamente correspondentes às três regras de formação de `ord1`

1. Caso `nil`: A lista está vazia. O lema `ord3_nil` resolve este caso rapidamente.
2. Caso de um elemento ($x :: \text{nil}$): A lista possui apenas um elemento. Novamente, o lema auxiliar `ord3_one` resolve isso direto.
3. Caso de dois ou mais elementos ($x :: y :: l$): Sabemos por hipótese que $x \leq y$ e que o restante da lista está ordenado sob `ord1`. É necessário abrir a definição de `ord3` e provar a ordenação para um índice genérico i , que pode ser
 - $i = 0$: É o primeiro elemento da lista. O comando `simpl` resolve as funções `nth`, revelando que precisamos provar que $x \leq y$. Como essa era a premissa fundamental da regra de `ord1`, o comando `assumption` fecha o subcaso.
 - $i > 0$: Comparação com elementos mais profundos na lista. O comando `simpl in *` destaca a sublista $y :: l$. `apply IHord1` (a hipótese de indução), garante que a sublista obedece à regra de índices. Os dois comandos `lia` finais apenas confirmam matematicamente que o índice atualizado não estourou o novo tamanho da lista.

Lemma `ord1_to_ord3` : $\forall l, \text{ord1 } l \rightarrow \text{ord3 } l$.

Lema `ord3_to_ord1` é indução sobre a estrutura da lista l , pois `ord3` não é indutivo. Primeiro, ao iniciar a lista e a indução, divide o problema em `l nil` ou lista $x :: y :: l$.

1. Caso `nil`: A hipótese diz que ela é `ord3`. Aplica-se `ord1_nil` direto para fechar o caso.
2. Caso de um ou mais elementos ($x :: l'$): `ord1` trata listas de um elemento de forma diferente de listas com 2 ou mais elementos. Por isso, necessário quebrar o caso em dois subcasos
 - Subcaso de um elemento ($x :: \text{nil}$): A lista é exatamente x . O comando `apply ord1_one` encerra a prova.
 - Subcaso de dois ou mais elementos ($x :: l''$): necessário aplicar a regra principal `apply ord1_all`.

Lemma `ord3_to_ord1` : $\forall l, \text{ord3 } l \rightarrow \text{ord1 } l$.

2.2.3 Lemas auxiliares para ord4

Para provar as equivalências finais envolvendo `ord4`, precisamos de duas pontes lógicas, detalhadas a seguir.

Lema `ord4_to_ord3` Este lema prova a implicação `ord4` $\rightarrow$ `ord3`. A lógica fundamenta-se no fato de que `ord3` é apenas um caso particular de `ord4`. Enquanto `ord4` garante que qualquer elemento é menor ou igual a todos os seus sucessores, a `ord3` exige apenas a verificação entre elementos adjacentes (índices i e $S\ i$). A prova se desenvolve da seguinte forma:

- Os comandos `unfold ord3`, `ord4` expõem as definições.
- Ao aplicar a hipótese H de `ord4`, precisamos apenas provar que o índice i é estritamente menor que seu sucessor $S\ i$.

Lemma `ord4_to_ord3` : $\forall l, \text{ord4 } l \rightarrow \text{ord3 } l$.

Lema `ord2_to_ord4` Este lema estabelece a implicação `ord2` $\rightarrow$ `ord4`. A prova é realizada por indução sobre a hipótese de ordenação `ord2` (tática `induction H`), dividindo o problema em dois casos principais:

1. Caso `nil`: Uma lista vazia é trivialmente `ord4`. Como não há elementos, os limites de tamanho da lista invalidam os índices, sendo resolvido com `lia`.
2. Caso $x :: l$: A análise depende da posição dos índices i e j . A tática `destruct i; destruct j` cria subcasos para verificar se estamos acessando o primeiro elemento (índice 0) ou elementos da cauda (índice > 0):

- Índices impossíveis: Se i ou j violam $i < j$ (ex: $0 < 0$), `lia` encerra por absurdo.
- Acesso cruzado ($i = 0$ e $j > 0$): Compara a cabeça x com um elemento da cauda. A definição de `ord2` garante que x é menor que tudo em l . Usamos o lema da biblioteca `nth_In` para associar o acesso por índice à presença do elemento na lista (`In`), satisfazendo a hipótese.
- Acesso na cauda ($i > 0$ e $j > 0$): A comparação ocorre inteiramente dentro da sublista l . O caso é resolvido invocando a hipótese de indução (`IHord2`).

Lemma `ord2_to_ord4` : $\forall l, \text{ord2 } l \rightarrow \text{ord4 } l$.

Lema `ord1_to_ord4` **Lemma `ord1_to_ord4`** : $\forall l, \text{ord1 } l \rightarrow \text{ord4 } l$.

Lema `ord4_to_ord1` **Lemma `ord4_to_ord1`** : $\forall l, \text{ord4 } l \rightarrow \text{ord1 } l$.

2.3 Teoremas principais

O objetivo principal desta proposta é mostrar que as definições `ord1`, `ord2`, `ord3` e `ord4` são logicamente equivalentes:

Theorem `ord1_equiv_ord2`: $\forall l, \text{ord1 } l \leftrightarrow \text{ord2 } l$.

Theorem `ord1_equiv_ord3`: $\forall l, \text{ord1 } l \leftrightarrow \text{ord3 } l$.

Theorem `ord1_equiv_ord4`: $\forall l, \text{ord1 } l \leftrightarrow \text{ord4 } l$.

Os três teoremas finais concluem as equivalências do projeto. Eles não necessitam de novas induções estruturais, pois podem reutilizar as pontes já estabelecidas anteriormente, aplicando a propriedade transitiva das implicações lógicas.

Teorema `ord2_equiv_ord3` Demonstra `ord2` $\leftrightarrow$ `ord3`.

- A ida faz o caminho lógico `ord2` $\rightarrow$ `ord1` $\rightarrow$ `ord3`.
- A volta faz o caminho lógico `ord3` $\rightarrow$ `ord1` $\rightarrow$ `ord2`.

Theorem `ord2_equiv_ord3`: $\forall l, \text{ord2 } l \leftrightarrow \text{ord3 } l$.

Teorema `ord2_equiv_ord4` Usamos os lemas auxiliares que acabamos de criar. Para a ida, aplicamos direto o lema `ord2_to_ord4`. Para a volta, fazemos o caminho longo: transformamos `ord4` em `ord3`, depois `ord3` em `ord1`, e finalmente `ord1` em `ord2`. **Theorem `ord2_equiv_ord4`:** $\forall l, \text{ord2 } l \leftrightarrow \text{ord4 } l$.

Teorema `ord3_equiv_ord4` Seguimos a mesma lógica de trânsito entre as provas. Para a ida (`ord3` $\rightarrow$ `ord4`), convertemos `ord3` para `ord1`, `ord1` para `ord2`, e usamos o nosso lema `ord2_to_ord4`. A volta é direta com o lema `ord4_to_ord3`. **Theorem `ord3_equiv_ord4`:** $\forall l, \text{ord3 } l \leftrightarrow \text{ord4 } l$.

3 Conclusão

Todas as equivalências propostas entre `ord1`, `ord2`, `ord3` e `ord4` foram demonstradas, comprovando que as abordagens indutivas e as abordagens baseadas em índices definem de maneira idêntica o conceito de ordenação de listas.